



Machine Learning Project

Chu Amélie 21206329

Tambellini Alan 21203966

Year 2025-2026

UM4IN811

Contents

1	Introduction	2
2	Dataset description	2
2.1	train_audio/	2
2.2	train.csv	2
2.3	train_soundscapes/	3
2.4	train_soundscapes_labels.csv	3
2.5	taxonomy.csv	3
3	Exploratory Data Analysis	4
3.1	Sites	5
3.2	Audio characterization	6
4	Classification task and evaluation	7
4.1	Outliers	7
4.2	Features	7
4.3	Train and test split	8
4.3.1	Data augmentation	9
4.4	Metrics and Models	10
4.5	Preprocessing	11
5	Experiments	12
5.1	Global Feature-Based	12
5.1.1	SVC train audio test results	13
5.1.2	SVC soundscapes test results	14
5.2	Chunk Aggregation Strategy	15
5.2.1	XGBoost and LR audio test results	16
5.2.2	XGBoost and LR soundscapes test results	17
5.3	Experiments and future work	19
5.3.1	Chunk Concatenation	19
5.3.2	Chunk Features Summary	19
5.3.3	Hierarchical Two-Stage Classifier	19
6	Conclusion	20
7	Annexe	21
7.1	Grid search best parameters	21

1 Introduction

This project is based on the **BirdCLEF+ 2026**¹ Kaggle competition, whose goal is to develop machine learning models capable of identifying bird species from audio recordings collected in the Brazilian Pantanal, South America.

In our project, we are constrained from using neural network models and instead rely only on classical machine learning methods. The task is a multi-label classification problem, where each recording can be associated with multiple species simultaneously.

2 Dataset description

The **BirdCLEF** database contains audio data from two sources, sampled at 32 kHz, enriched by users of **xeno-canto**² (XC) and **iNaturalist**³ (iNat). It is divided into several folders:

2.1 train_audio/

This folder contains 206 subfolders, each corresponding to a species (**primary_label**). These subfolders contain audio recordings of a single species, with sometimes **secondary_labels** corresponding to other species that are also present in the recording, totaling 35,549 audio files in **.ogg** format. The file names in each subfolder correspond to the recording names.

2.2 train.csv

This file contains the metadata of the **train_audio** files and includes 15 columns across 35,549 rows (with 206 unique primary labels), as shown in Table 1. Both primary and secondary labels appear in the set of classes to predict (in **taxonomy.csv**). It should be noted that all secondary labels also appear as primary labels, but not all primary labels appear as secondary labels.

Column Name	Description
primary_label	Code of the main observed species
secondary_labels	List of other species present in the audio
type	Type of recorded sound (e.g., song, call, noise)
latitude	Latitude of the recording location
longitude	Longitude of the recording location
scientific_name	Scientific name of the species
common_name	Common name of the species
class_name	Taxonomic class of the species
inat_taxon_id	Taxonomic identifier on iNaturalist
author	Author of the observation
license	Type of license associated with the recording
rating	Audio quality: 1 (low) to 5 (high), 0 n.a. ⁴ ; step 0.5 ⁵
url	URL leading to the original recording
filename	Name of the file associated with the recording
collection	Source of the collection: XC or iNat

Table 1: Description of the **train.csv** file

¹ <https://www.kaggle.com/competitions/birdclef-2026> ² <https://xeno-canto.org/> ³ <https://www.inaturalist.org/> ⁴ for Xeno-canto, rating is reduced by 0.5 if background species are present ⁵ For iNat collection, there are no available ratings

2.3 train_soundscapes/

This folder contains soundscape audio recordings that may include multiple species, with a total of 10,658 recordings. The files follow a naming convention of the form:

BC2026_Test_<file ID>_<site>_<date>_<time in UTC>.ogg

2.4 train_soundscapes_labels.csv

This file contains 1,478 rows, corresponding to 66 unique audio recordings from **train_soundscapes** that have been labeled by experts, representing approximately 0.6% of the soundscape files. It includes the four columns shown in Table 2.

Column Name	Description
filename	Name of the file associated with the recording
start	Start of the 5-second segment
end	End of the 5-second segment
primary_label	List of primary labels of the species heard in the segment

Table 2: Description of the **train_soundscapes_labels.csv** file

2.5 taxonomy.csv

This file contains 234 rows corresponding to the 234 classes to be predicted in the submission. The columns are shown in Table 3. To better understand the column structure, Figure 1 presents the hierarchical representation of the classes.

Column Name	Description
primary_label	Code of the main observed species
inat_taxon_id	Taxonomic identifier on iNaturalist
scientific_name	Scientific name of the species
common_name	Common name of the species
class_name	Class of the species

Table 3: Description of the **taxonomy.csv** file

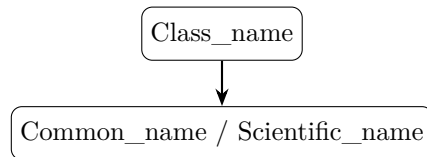
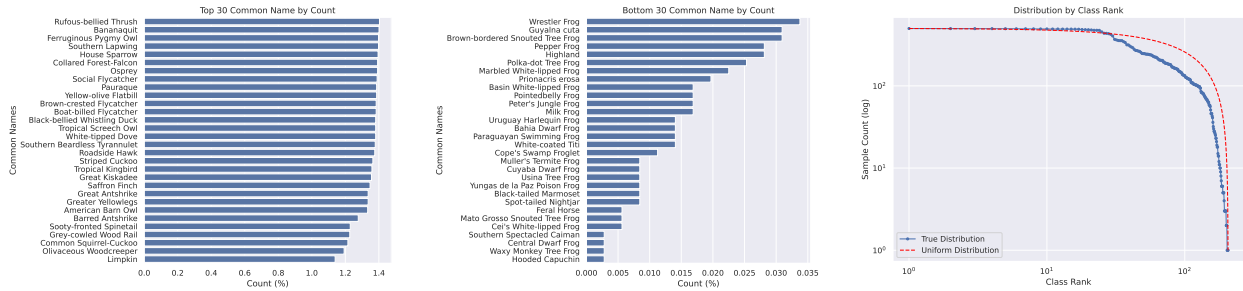


Figure 1: Hierarchical classification of a species.

3 Exploratory Data Analysis

We first explore the data. Since the main data to be processed consists of audio recordings, we also need to work directly with audio waveforms. To do so, we investigate Python libraries suited for audio processing. The primary library used is `librosa`, which is widely adopted for machine learning applications on audio data. We also considered useful libraries listed in this GitHub repository⁶.

The first observation concerns the imbalance in the label distribution. Figure 2 illustrates this imbalance through the most and least frequent common names. The distribution by class rank further highlights it when compared to an ideal uniform distribution.



(a) Most Frequent Common Name. (b) Less Frequent Common Name. (c) Distribution by Class Rank.

Figure 2: Distribution of common name frequencies and their distribution by class rank.

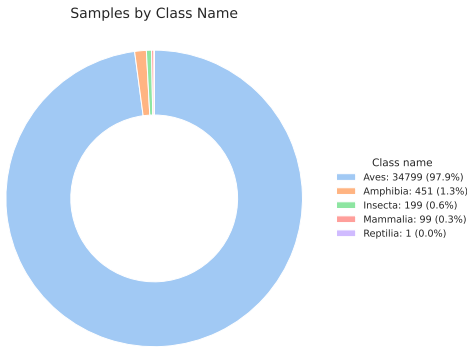
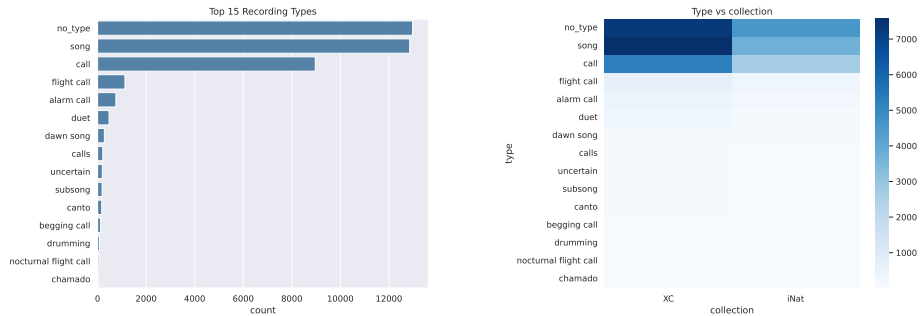


Figure 3: Class name distribution.

As we have seen that the distribution of the labels was imbalanced, we can also see that in terms of class name, there is a prevalent class, the class *Aves*, as shown in Figure 3.

Among the different variables available, not all will be used, as we first examine their distributions and relevance. Figure 4 shows the distribution of recording types. Only a few categories are widely represented. A closer inspection reveals inconsistencies in the labels: similar categories appear under different forms (e.g., *call* and *calls*), as well as in other languages (e.g., *chamado*, which means “call” in Portuguese). This variable is therefore noisy, inconsistent, and not particularly informative.



(a) Across the dataset. (b) Per collection.

Figure 4: Recording types distribution.

⁶ <https://github.com/andreimatveyeu/awesome-python-audio>

We also observe in Figure 5 the geographic distribution of species recordings. The data is globally distributed, with a majority of recordings located in the Americas, and a broad spatial spread with an approximate standard deviation of around 20° in both latitude and longitude. Figure 6 shows the distribution of audio durations. We observe that most recordings are around 150 seconds long, with a smaller proportion of very short recordings, some lasting less than one second.

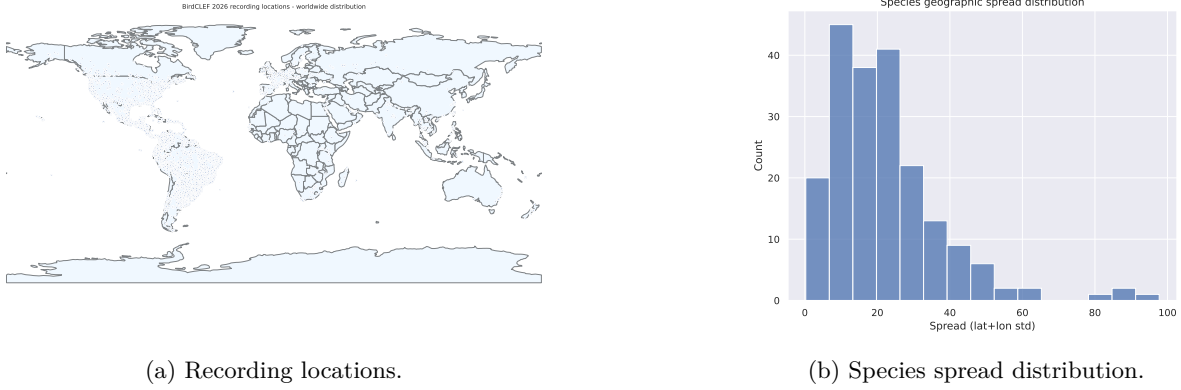


Figure 5: Global distribution of recordings and species spread across locations.

3.1 Sites

As seen in section [train_soundscapes/](#), the filename contains the site of the recording. As the test set also follows the same naming convention, it could be interesting to see whether the site could be an indication for the task. For this, for each soundscape recording, we retrieve the labels associated with it; then from the train audio where we have the location, we map the locations to this site.

In Figure 7a, we show the site centroids obtained when taking all labels. Note that some labels appear in more than one site. Figure 7b displays sites without these multi-site labels. We can see some sites disappear, but altogether, the sites do not correspond to one unique concentrated place; points are scattered across countries and continents, so we can omit this information.

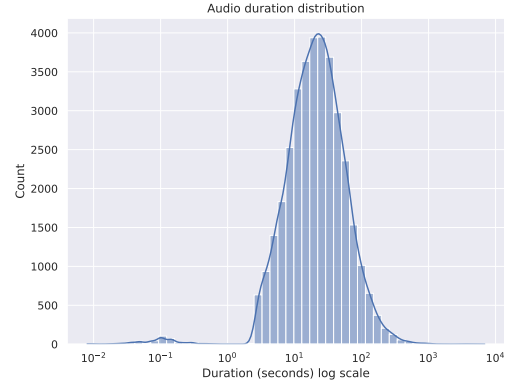


Figure 6: Durations distribution of audios.

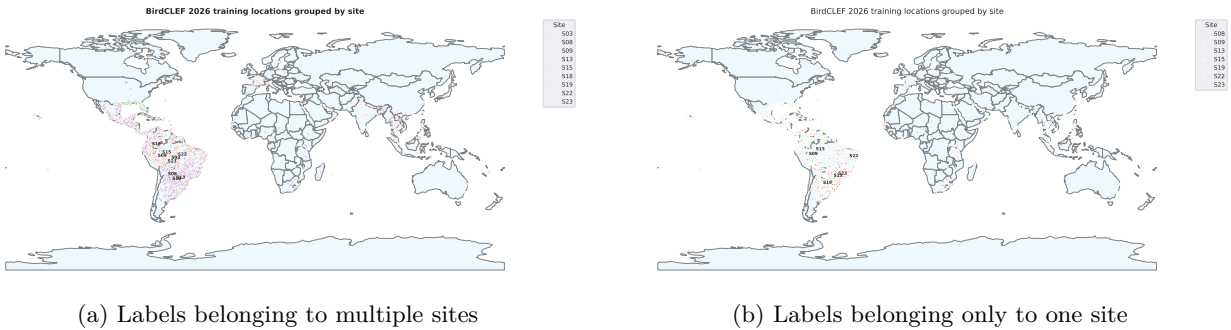


Figure 7: Comparison of recording sites based on label density.

The dataset includes the Pantanal recording location for the soundscape audio, as shown in Figure 8. As expected, within this small area, the sites are scattered without any apparent structure, not really providing

discriminative information.

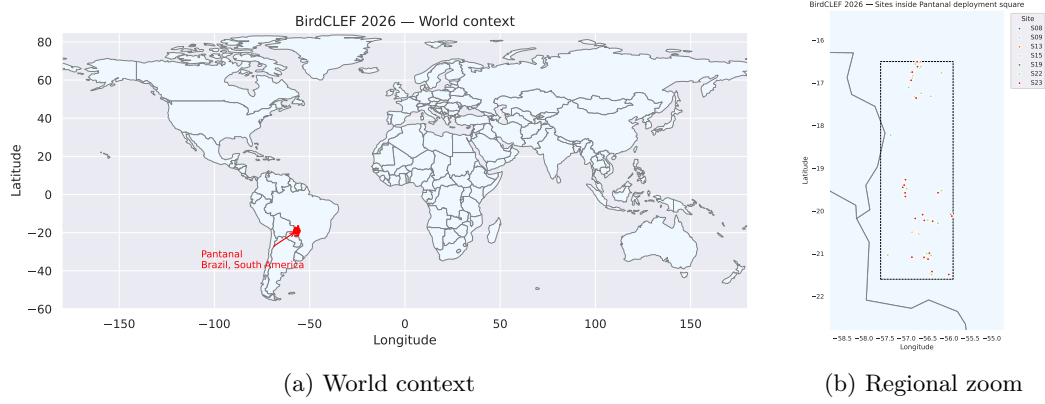


Figure 8: Geographic location of the Pantanal study area.

3.2 Audio characterization

We explore the features we can extract from the audios. Each animal class has distinct audio features. Birds produce sharp, high-frequency calls. Frogs occupy a low-frequency area. Insects generate continuous noise. Thus, we try to find features that we can use for our training.

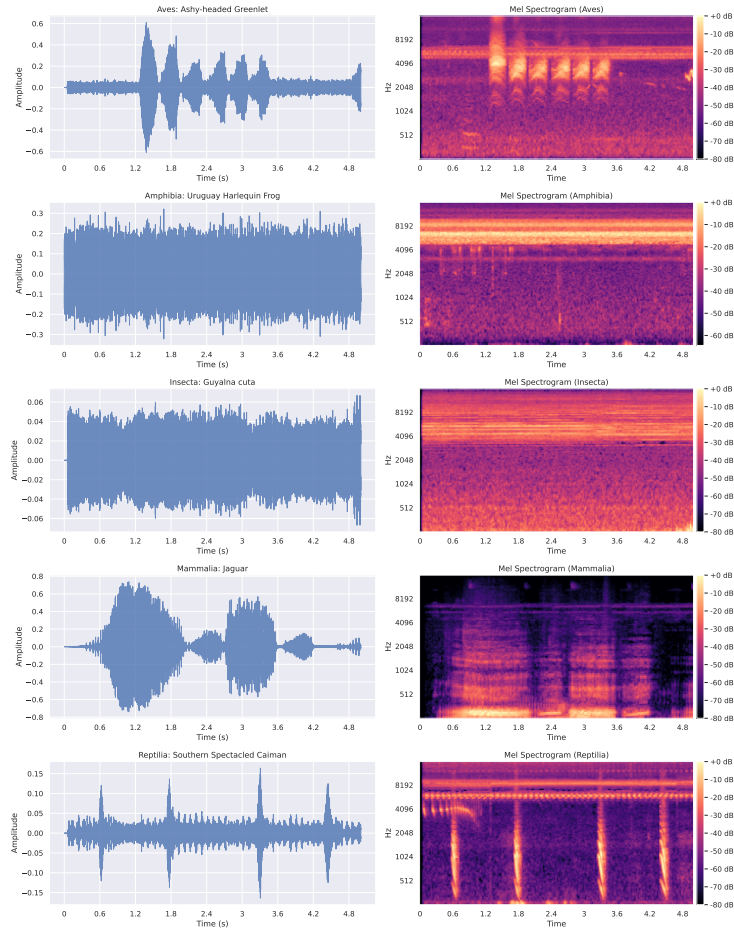


Figure 9: Waveform and Mel-spectrogram per class name. We pick one representative species from each class.

Figure 10 shows the average decibel levels across frequency bands for each class, computed over class-specific subsets. Some differences can be observed in the distribution of energy across frequencies, indicating variations in where sound intensity is concentrated. It should be noted that the Reptilia class contains only a single sample and is therefore not representative. Overall, certain species exhibit higher intensity in specific frequency ranges.

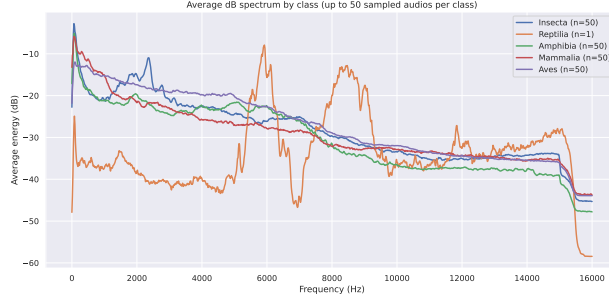


Figure 10: Frequency distribution average per class.

4 Classification task and evaluation

For this classification task, we have two distinct training datasets: one consisting of audio recordings of a target species annotated with secondary species labels, and another composed of soundscape recordings segmented into 5-second clips, each labeled with the species present in that interval.

A possible strategy is to initially train the models on the target-species audio dataset to learn species-specific patterns and subsequently fine-tune them on the soundscape dataset, where environmental noise is more prominent. Since the Kaggle test set follows the same structure as the soundscape data, prioritizing this dataset in the later stages of training is a coherent choice. We also considered concatenating the 5-second soundscape segments into a single audio file per original recording. However, this would reduce the number of training samples from 1,478 to 66 and would no longer match the format of the true test data.

In our experiments, we disregard the secondary labels in the target-species dataset and train the models solely using the primary labels. The final output consists, for each soundscape recording, of a probability vector over all classes.

4.1 Outliers

Audio duration shows high variance across recordings, as shown in Figure 6. To limit the influence of very short or very long recordings, we remove duration outliers using the 1st and 99.5th percentiles as bounds. However, some labels have very few samples. If we removed all their outliers, it could drop them below the rare label threshold of 6 samples, making cross-validation unreliable. Therefore, we apply a constrained removal: for each label, we remove as many outliers as possible while retaining at least 6 samples, prioritizing the longest durations first. Labels already at or below the threshold are left untouched.

Additionally, globally rare labels, meaning those with fewer than 6 samples before any removal, are excluded from outlier removal entirely. They will be handled by data augmentation in Section 2.3.

4.2 Features

From the raw waveform obtained via **librosa**, we extract a set of acoustic features that serve as input representations for our tasks. Since these features are computed as matrices over time frames, we **aggregate** them into fixed-length 1D vectors using five statistics:

$$x_{\text{feat}} = [\text{mean}, \text{std}, p_{10}, p_{90}, \text{max}]$$

The mean and standard deviation capture the overall distribution of the feature, while p_{10} provides a noise-robust baseline and p_{90} highlights strong acoustic events. The maximum is included to preserve short bursts,

like chirps, that may be diluted by the mean when they occur only briefly.

The extracted features are as follows:

- **MFCC** (30 coefficients): Mel-Frequency Cepstral Coefficients capture the spectral envelope of the signal in a compact representation that approximates human auditory perception. We chose 30 coefficients as it seems a good middle ground.
- **Delta MFCC**: The first-order temporal derivative of the MFCCs, encoding the rate of change of the spectral envelope over time. This is particularly useful for distinguishing species with similar pitch but different temporal modulation patterns.
- **Delta-Delta MFCC**: The second-order temporal derivative, capturing the acceleration of spectral changes. It is well-suited to characterising rapid vocalisation patterns such as chirps and trills.
- **Spectral Centroid**: The weighted mean of the frequencies present in the signal, interpreted as the perceived brightness of a sound. A high centroid corresponds to a bright, sharp timbre, while a low centroid indicates a dull, low-frequency character.
- **Spectral Contrast**: Measures the difference in amplitude between spectral peaks and valleys across sub-bands, reflecting the harmonic richness of the signal. High contrast is associated with tonal sounds, while low contrast is characteristic of noise-like signals.

Finally, the different feature vectors are concatenated to build the final representation:

$$x = [x_{\text{MFCC}}, x_{\Delta\text{MFCC}}, x_{\Delta^2\text{MFCC}}, x_{\text{SC}}, x_{\text{SCont}}] \in \mathbb{R}^{490}$$

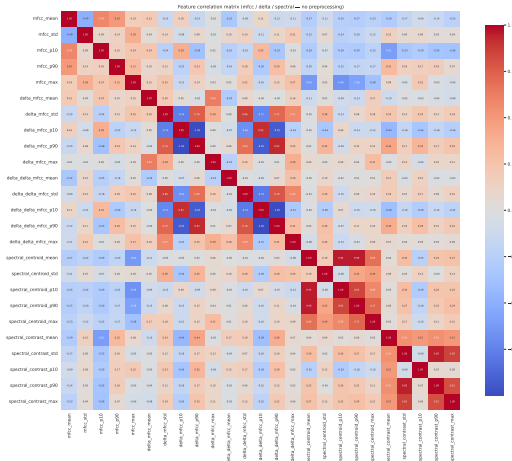


Figure 11: Feature correlation matrix.

Figure 11 presents the correlation matrix for a subset of these features. Several features appear to be significantly correlated. However, since this analysis was performed after training, and given the high computational cost of retraining, we did not conduct additional experiments with correlated feature removal.

4.3 Train and test split

As both datasets are different, they require their own splitting strategy. For the train audio set, we use a stratified split to keep the label distribution balanced across folds. For the soundscape audio, samples can belong to the same recording file, so we use a **StratifiedGroupKFold** grouped by filename to avoid data leakage. We do not stratify by primary label, as each sample can have multiple labels and some of them appear in only one file, making it impossible to stratify and group by filename at the same time.

As shown in Figure 12, some labels in the train soundscapes set appear in only one audio file. As a result, those labels will necessarily end up entirely in either the training or the testing fold.

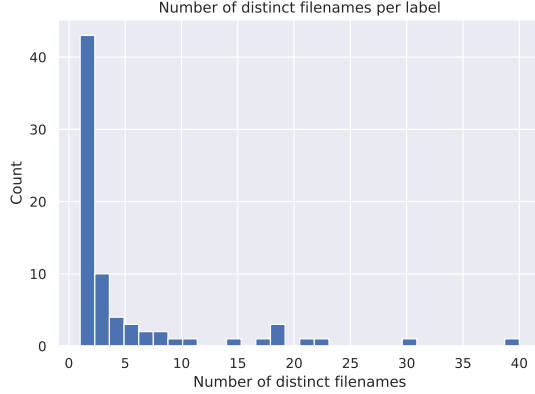


Figure 12: Train soundscapes labels distribution over filenames.

Our experiments are organized as follows:

- We first split the train audio dataset into training and testing sets. We then perform 5-fold cross-validation on the training set, together with hyperparameter tuning, to select the best-performing model.
- Using the selected model, we conduct two experiments. First, we evaluate the model directly on the full soundscapes dataset to measure its generalization performance. While some training samples contain overlapping species, the soundscapes dataset presents a more challenging acoustic environment with stronger background noise and more complex mixtures. We therefore on a second time, fine-tune the model on a train split of the soundscapes dataset and evaluate it on the corresponding test split. Because some splits may favor certain labels, we repeat the evaluation with 10 different random seeds. This allows us to compare the model’s performance with and without fine-tuning more reliably.

4.3.1 Data augmentation

For the train audio set, we train our models using a $k = 5$ cross-validation split. Some labels have very few samples; 18 out of 206 classes have fewer than 6 samples, representing 8% of the labels, which makes them difficult to evaluate reliably in a cross-validation setting.

For these rare classes, we apply data augmentation: for each class, one original sample is held out in the test fold, and the remaining original samples are placed in the train fold. We then generate augmented versions of the train samples up to a total of 6, by randomly applying a combination of the following 6 transformations shown in Figure 13. As the combination is chosen randomly, to ensure there is at least one transformation, we always apply masking.

Since the augmented samples are derived from the original recordings, they are, by construction, correlated with them; the augmentation is therefore not meant to improve generalization in a strict sense, but rather to allow the model to see these rare classes during training. Predictions on these classes should be interpreted with caution: if their metrics appear overly optimistic, we will exclude them from the global evaluation summary.

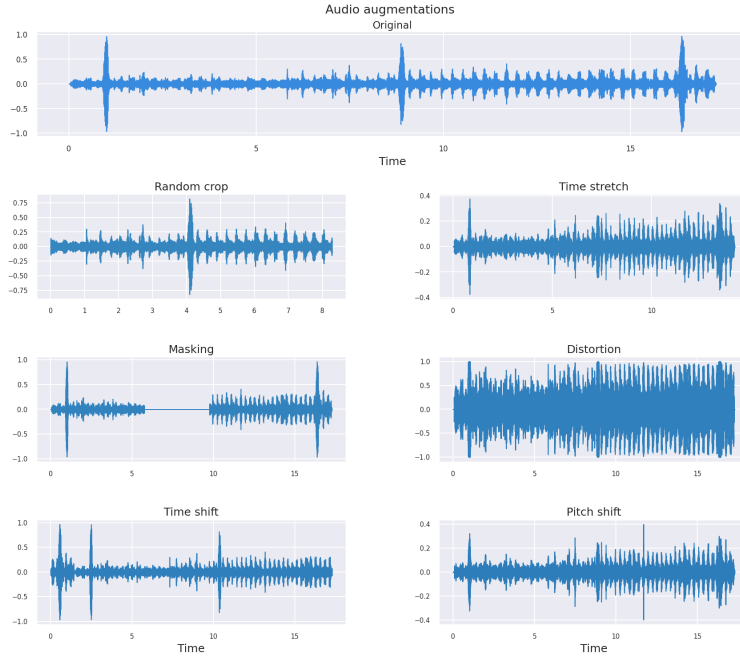


Figure 13: Example data augmentation on audio blttit1/XC711283.ogg

4.4 Metrics and Models

The metric used in Kaggle is the macro-averaged ROC AUC. Since the train set contains single-label samples and the soundscapes contain multi-label samples, we evaluate both splits with a dedicated set of metrics.

Audio train	Audio soundscapes
Accuracy top k	LRAP
ROC AUC	ROC AUC
Macro F1	Macro F1
macro AP	macro AP

Table 4: Evaluation metrics used for each dataset.

For the single-label audio data, we use top- k accuracy to measure whether the correct class appears among the highest predictions, along with macro F1 to account for class imbalance. ROC AUC and macro average precision are also reported to evaluate the quality of probabilistic separation between classes.

For the multi-label soundscapes, we use LRAP (Label Ranking Average Precision) as the primary ranking-based metric, since it is robust to class imbalance and reflects how well true labels are ranked above irrelevant ones. Macro F1, ROC AUC, and macro average precision are also used to provide complementary views of classification and ranking performance across classes.

Regarding models, we restrict ourselves to classical non-neural models: Logistic Regression, Support Vector Machines (SVC), and XGBoost, to provide strong and interpretable baselines under limited computational complexity.

All models are trained using a One-vs-Rest (OvR) strategy, where one binary classifier is learned per class. This formulation is well-suited to both single-label and multi-label settings, as it allows independent probability estimation for each class without enforcing mutual exclusivity. Since OvR classifiers produce independent scores per class, we do not assume a shared decision threshold. Instead, we tune a class-specific threshold by maximizing the F1-score on out-of-fold validation predictions, which improves robustness under severe class imbalance.

4.5 Preprocessing

Listening to a sample of the recordings, we noticed significant variability in audio quality: some clips contained background noise, others had long silent segments, and the target species could be either faint or dominant in the mix. This led us to consider several preprocessing steps:

- **Trimming**: removing leading and trailing silence below a chosen dB threshold
- **Bandpass filter**: keeping only frequencies within a biologically relevant range
- **Noise removal**: attenuating background noise
- **RMS normalisation**: scaling each clip to a target loudness level

However, most of these methods require careful tuning of parameters that are hard to set globally. A frequency cutoff or noise threshold that works well for one recording may suppress the target signal in another, depending on the species or the recording conditions. Whether such thresholds should be set globally or per class adds further complexity, and since audio quality is independent of the label, a poorly chosen value could harm more classes than it helps.

We therefore chose to start applying only **RMS normalisation**, as it requires no such tuning: it simply scales each recording to a consistent loudness level, reducing the effect of volume variability across clips without making any assumption about the content of the audio.

$$y_{norm} = \frac{y}{\sqrt{\frac{1}{N} \sum_{i=1}^N y_i^2}}$$

As shown in Figure 14, this linear transformation standardizes the average energy across the dataset. This ensures that the model learns features based on spectral morphology rather than arbitrary volume variability or microphone distance.

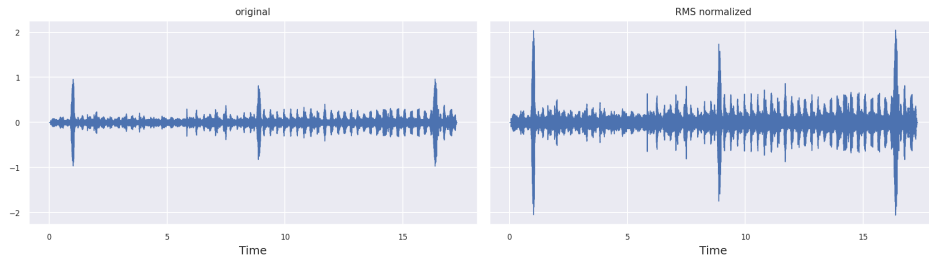


Figure 14: RMS normalization on audio `blttit1/XC711283.ogg`

5 Experiments

We implement our models using different strategies. We report the results obtained from training on the train audio dataset to evaluate the overall performance of our models, but the main results of interest are those obtained from the train soundscapes dataset.

Regarding the features, we could have explored different combinations to identify the most effective ones. However, since such experiments would have been computationally expensive, we chose to use the feature set described in Section [Features](#) consistently across all our strategies. Throughout the project, we evaluated two different approaches.

The **first** strategy is a global feature-based approach, where features are extracted from the entire audio recording as a whole.

The **second** strategy is a chunk aggregation approach. In this setting, each audio recording is divided into overlapping windows, and features are extracted independently from each chunk. Final predictions are then obtained by aggregating the predictions across all chunks. We initially use max aggregation, under the assumption that if a species is detected in at least one chunk, then it is present in the complete recording. This approach allows us to better capture temporal information within the audio signal.

5.1 Global Feature-Based

Figure [15](#) illustrates the pipeline used for the first strategy. We perform a grid search over the three selected models (see Appendix for the best hyperparameter configurations). For this approach, the chosen features are extracted from the full audio recording and aggregated using the mean, standard deviation, p10, p90, and maximum value. These aggregated descriptors are then concatenated to form the final vector representation of the audio recording. Table [5](#) presents the results obtained with this strategy. We first report the cross-validation scores for each model. After selecting the best-performing model, we evaluate it on the test set. Finally, we report its performance on the soundscape dataset.

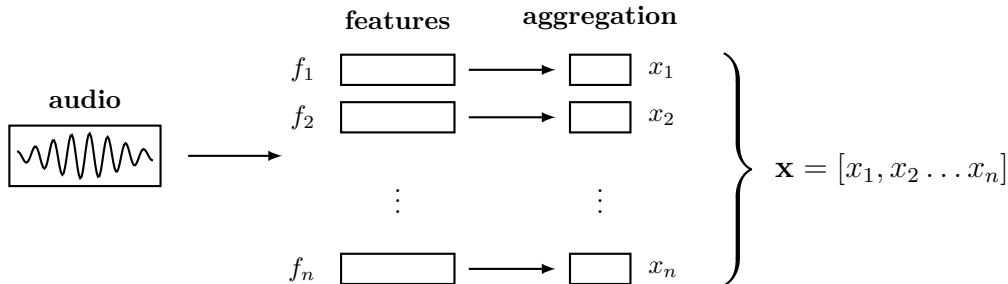


Figure 15: Audio global feature aggregation process.

Evaluation metrics are computed on non-augmented samples only, as augmented clips are synthetic variants of training recordings; including them inflates metrics due to class-level coarseness and fold leakage across OOF splits. In general, $\text{top-1} < 0.50$ confirms models cannot reliably identify the correct species as their first choice, which is expected given 150+ species with severe imbalance. We present our results both on the train and on the soundscape sets.

Table [5](#) shows the results of our models on the different datasets. We evaluated our models on the audio with cross-validation and, based on these results, chose the best model to evaluate on the test scores. We can note that our SVC model achieves slightly higher scores on the held-out test set than in cross-validation (Top-1: 0.4999 vs 0.4773, AP: 0.4425 vs 0.4168), suggesting the model generalizes well without overfitting to the training folds.

Audio CV scores				
Model	Accuracy (top 1)	F1	ROC AUC	AP
<i>LR</i>	0.4353	0.3780	0.9119	0.2817
<i>XGBoost</i>	0.3679	0.3039	0.9008	0.2738
<i>SVC</i>	0.4773	0.4433	0.9160	0.4168
Audio test scores				
<i>SVC</i>	0.4999	0.4502	0.9225	0.4425
Soundscapes test scores				
Model	LRAP	F1	ROC AUC	AP
<i>SVC</i>	0.2243	0.0067	0.5841	0.1862

Table 5: Models scores for Global Feature-Based strategy.

5.1.1 SVC train audio test results

We analyze the results of our best model, SVC, on the audio test set. As we can see in the graphs below, the SVC trained on global features shows strong taxonomic bias. The confusion matrix in Figure 16 shows that Mammalia and Amphibia are predominantly misclassified as Aves, reflecting the severe class imbalance in the dataset. This is also confirmed by the PR curves in Figure 17, where Aves achieves almost perfect precision across all recall levels, while Mammalia collapses at recall ≈ 0.5 and Amphibia degrades gradually. This suggests that the model fails to maintain precision for minority classes beyond moderate recall. Insecta is the best-performing non-Aves class, as shown in the PR and higher ROC AUC curves.

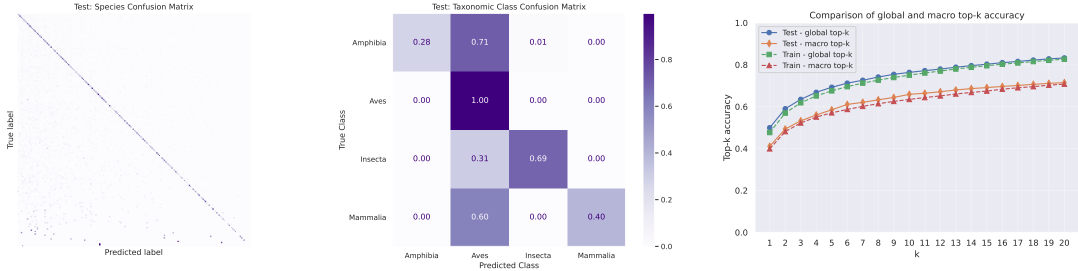


Figure 16: Confusion matrix per label and class and top k accuracy.

Furthermore, the AUC-AP discrepancy (macro ROC AUC ≈ 0.92 vs. macro AP ≈ 0.42) confirms that the model predicts positives correctly in aggregate but lacks precision when confident, meaning when the model is confident, it is often still wrong. This pattern is typical of imbalanced datasets: AUC is inflated by the large number of true negatives (easy to rank below positives), while AP is penalized by the many false positives.

Top- k accuracy shows global scores consistently above macro and test marginally above train across both. This indicates that the model captures taxonomic structure but struggles with fine-grained species differentiation. This is a major problem, probably caused by feature overlap and class imbalance rather than overfitting. However, increased gain from Top-1 to Top-5 indicates the correct species is frequently among the top candidates, but the model does not have enough confidence to rank it first.

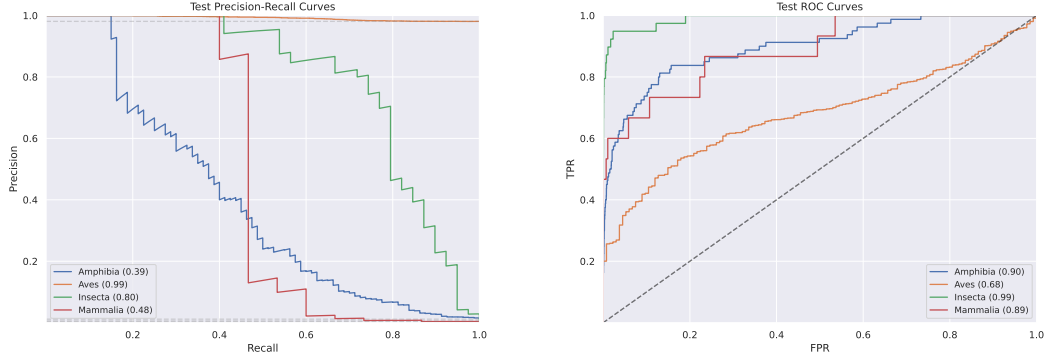


Figure 17: Precision-recall and ROC AUC curves.

Finally, in Figure 18, we don't see any strong positive correlation between AP and label frequency (however, when frequency decreases, AP follows marginally), which suggests the problem is not just data quantity, but also feature discriminability and class overlap. Several solutions are possible: oversampling rare species through augmentation, undersampling Aves to fight against the class majority.

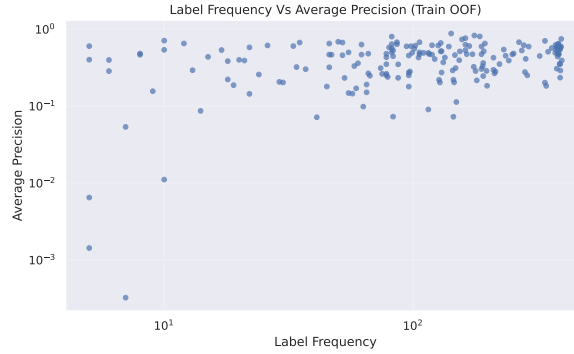


Figure 18: Average precision per label frequency.

5.1.2 SVC soundscapes test results

Using our SVC model, we then move on to evaluating it on the soundscapes test set.

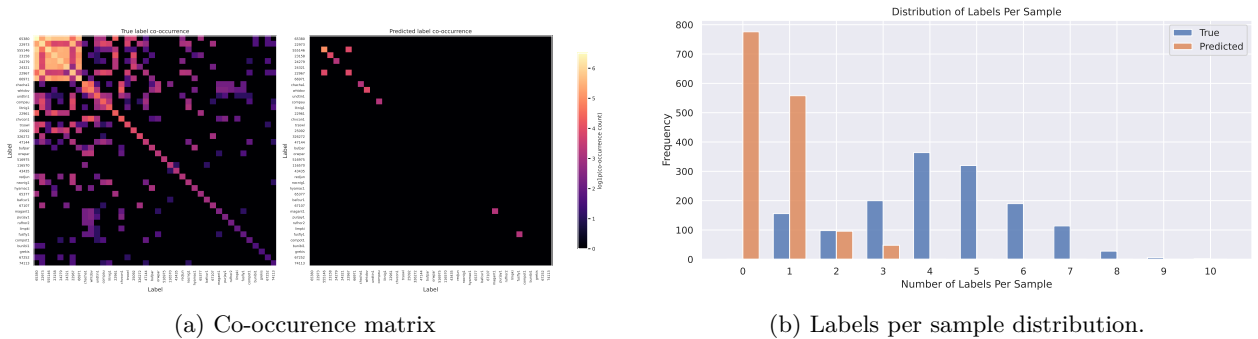


Figure 19: Soundscapes results SVC.

Figure 19 presents the predicted labels on the soundscape audio dataset. The co-occurrence matrix shows, for each pair of labels, how often they appear together in the ground truth annotations. The frequencies are displayed on a logarithmic scale to account for label imbalance.

We can observe that the model predicts a large number of empty label sets, as reflected by the nearly empty co-occurrence matrix. However, the label-per-sample distribution indicates that the model is still capable of predicting multiple labels for a single sample. Nevertheless, predictions containing a high number of labels are still rare, with their frequency decreasing as the number of labels to predict increases.

The results obtained on the soundscape dataset indicate that the SVC model struggles to generalize effectively to the more complex and noisy environment. Although the ROC AUC score of 0.5841 suggests that the model retains some ability to discriminate between labels, the low F1-score and AP reveal that the final binary predictions are rarely correct. This behavior is consistent with the observed tendency of the model to predict many empty label sets.

Similarly, the LRAP score suggests limited ranking performance, indicating that relevant labels are not consistently assigned high confidence scores.

With the baseline, we consider that the whole audio corresponds to the label, which isn't the case, as some of the species can appear in a fraction of the audio. Thus, instead of aggregating our whole audio into one vector, we represent the time in the audio with window chunks.

5.2 Chunk Aggregation Strategy

In this approach, each audio recording is divided into overlapping 5-second chunks. Each chunk is represented by a feature vector and assigned the same primary labels as the original recording. To avoid associating silent or noisy segments with the labels, chunks whose RMS energy falls below a threshold are discarded. Since the audio is RMS-normalized, we set this threshold to 0.01.

The model is trained independently on each chunk. During inference, chunk-level predictions are aggregated to obtain a recording-level prediction. We first use max aggregation, under the assumption that if a species is detected in at least one chunk, then it is present in the full recording. Figure 20 illustrates this strategy. Since chunking the audio recordings generates a significantly larger number of samples and feature extraction is computationally expensive, we do not perform an additional grid search. Instead, we reuse the best hyperparameters identified in the previous strategy.

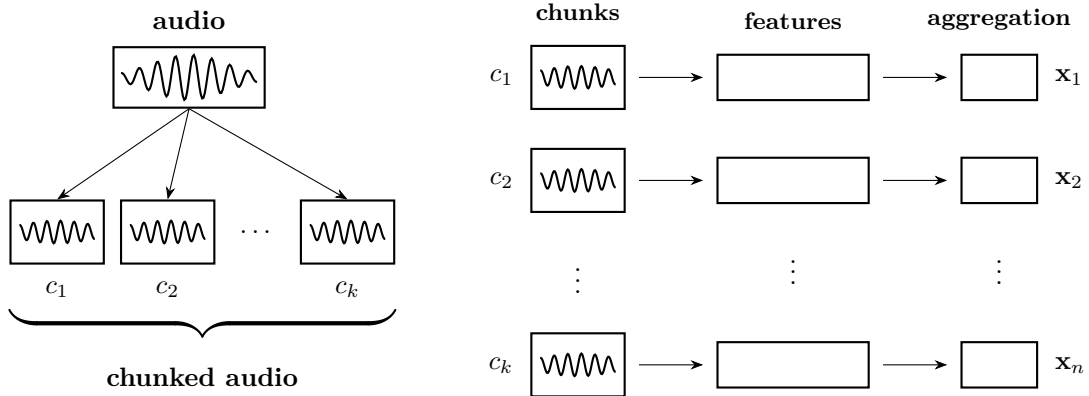


Figure 20: Audio chunk aggregation process.

Table 6 shows the scores for each model we tested. We can observe that the SVC model obtained the lowest scores. These results are explained by the fact that we set `max_iter` to 50, as setting it to 500 took too long to compute the results compared to the previous strategy: for the train audio set only, we already have 210k chunks for 33k samples. In the Table, we put the results for the 2 best models. If XGBoost seemed to be the best model based on the metrics, by later comparing the co-occurrence matrix and the plot results on the soundscapes dataset, we noticed that our XGBoost model didn't perform as well as compared to LR, hence the choice of displaying in this section some results of the LR model.

Audio CV scores				
Model	Accuracy (top 1)	F1	ROC AUC	AP
<i>LR</i>	0.3202	0.2982	0.8866	0.2423
<i>XGBoost</i>	0.4305	0.3614	0.9077	0.3406
<i>SVC</i>	0.0260	0.0528	0.7037	0.1149

Audio test scores				
<i>LR</i>	0.3253	0.2846	0.8934	0.2528
<i>XGBoost</i>	0.4572	0.3701	0.9104	0.3737

Soundscapes test scores				
Model	LRAP	F1	ROC AUC	AP
<i>LR</i>	0.1531	0.0069	0.6731	0.1878
<i>XGBoost</i>	0.1616	0.0018	0.6467	0.1731

Table 6: Models scores for Chunk Aggregation Strategy.

5.2.1 XGBoost and LR audio test results

We analyze the results of our LR and XGBoost models on the test set. In Figures 21 and 22, we can notice, in terms of the confusion matrix, that for the labels at the right of the matrix, the diagonal is more pronounced in the LR model compared to XGBoost’s matrix. The confusion matrix at the class level of both models is similar: they both struggle with under-represented classes.

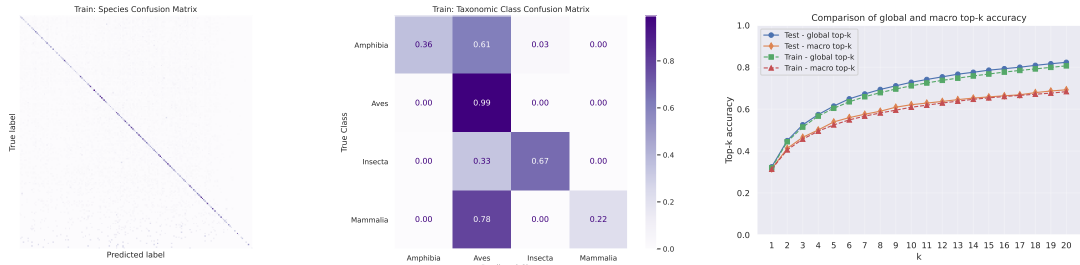


Figure 21: Confusion matrix per label and class and top k accuracy LR.

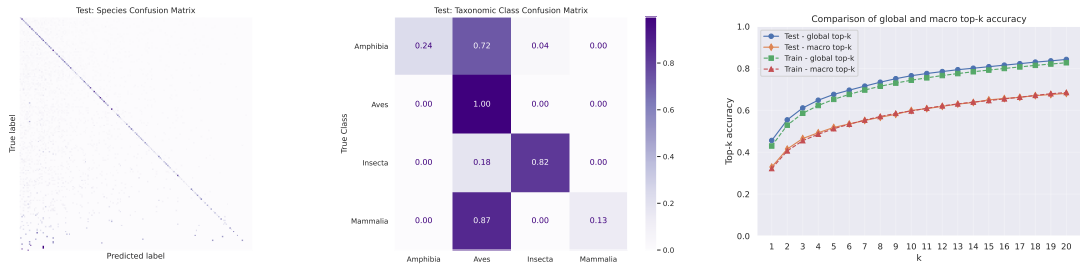


Figure 22: Confusion matrix per label and class and top k accuracy XGBoost.

The XGBoost model trained on chunk-level features shows an even stronger Aves bias than the SVC: the confusion matrix presents the collapse of Mammalia, Amphibia, and Reptilia into Aves, and in Figure 23 and 24, the PR curve confirms this with Aves at 0.99 while all minority classes drop sharply at low recall

thresholds much faster than with the SVC. XGBoost also shows a clearer positive relationship between label frequency and AP, suggesting it relies more heavily on sample count to build discriminative trees, making it more sensitive to imbalance.

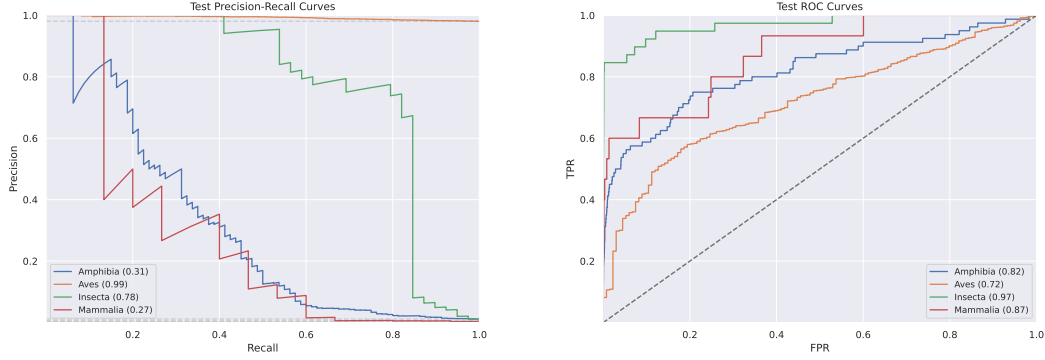


Figure 23: Precision-recall and ROC AUC curves for XGBoost.

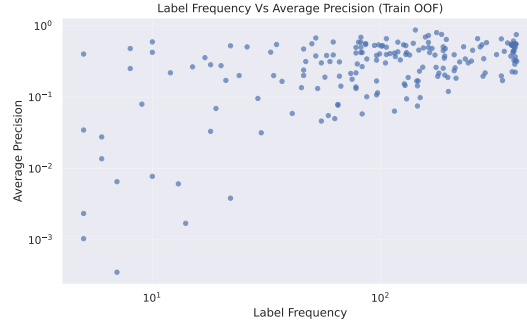


Figure 24: Average precision per label frequency XGBoost.

Comparing XGBoost to LR, we see in Figure 25 that the two models show opposite confidence behaviours. XGBoost produces poorly calibrated, low-confidence predictions as most scores concentrate near 0 with very few near 1, and over 25% of predictions have a near-zero gap between the top-1 and top-2 scores, indicating the model is frequently uncertain between competing species. LR, by contrast, is overconfident: $\approx 80\%$ of its predictions is near probability 1, with a near-zero top-1/top-2 gap, meaning LR almost always assigns near-certain probability to a single class. Both behaviours are problematic: XGBoost lacks decisiveness while LR is miscalibrated toward certainty, likely defaulting to Aves with high confidence in most cases.

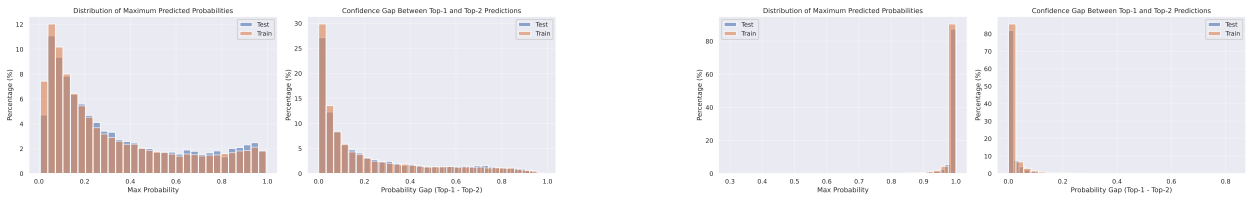


Figure 25: Max probabilities distribution and Confidence Gap XGBoost (left) LR (right).

5.2.2 XGBoost and LR soundscapes test results

As we have seen in the audio results, our LR model had higher predicted maximum probabilities for our data, so we plotted the results for both of the models. Figure 26 and Figure 27 show for LR and XGBoost the co-occurrence matrix of the predicted and true labels, as well as the labels per sample distribution.

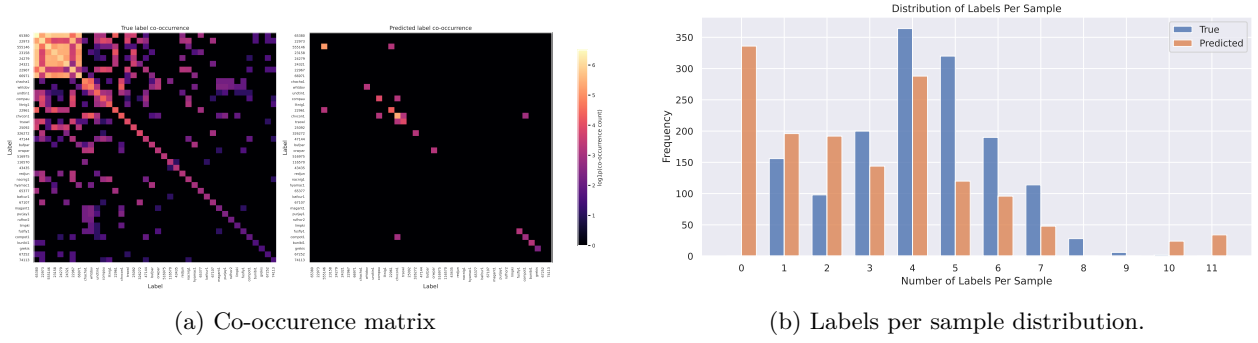


Figure 26: Soundscapes results LR.

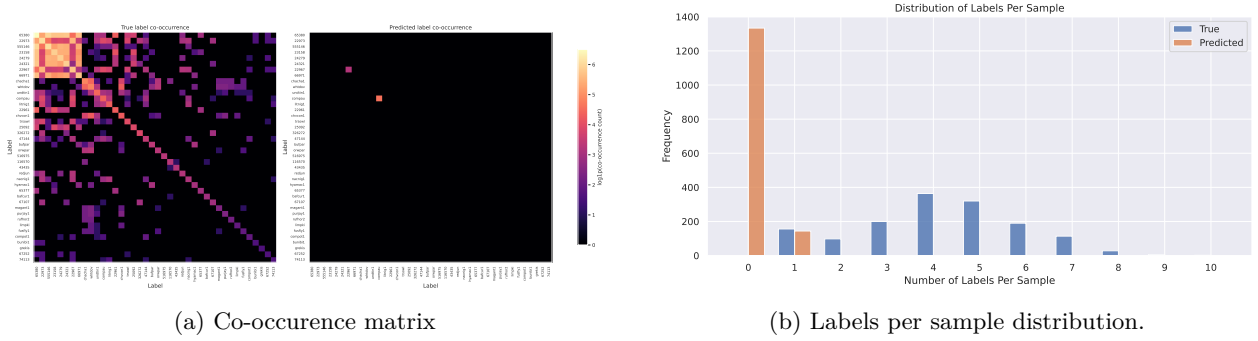


Figure 27: Soundscapes results XGB.

We observe that the XGBoost model struggled significantly to predict labels on the noisier dataset. Most sample predictions were empty, resulting in an almost empty co-occurrence matrix. Moreover, when the model predicted a label, it generally predicted only a single label rather than multiple labels simultaneously. While LR was able to predict multiple labels per sample more effectively, it still faced difficulties and produced empty predictions for some samples. Among all the models, LR exhibited the most balanced label-per-sample distribution. This suggests that the higher evaluation metrics achieved by XGBoost may partly come from its tendency to avoid classification on the soundscape test set.

When compared with our best SVC model from the previous strategy in Figure 19, the SVC appears to perform between XGBoost and LR. Although it also produces many empty label predictions, it is less prone than XGBoost to failing when predicting multiple labels simultaneously.

We also notice, when analyzing the tables, that our SVC model achieves a higher LRAP score (0.2243) compared to the two others (LR: 0.1531, XGBoost: 0.1616), but obtains a lower ROC AUC (0.5841) against them (LR: 0.6731, XGBoost: 0.6467). This indicates that SVC, which relies on decision function scores rather than calibrated probabilities, tends to produce sharper and more concentrated scores for the most relevant labels, improving the ordering of the top predictions and therefore benefiting LRAP. However, this same characteristic limits how well the scores reflect global class separability across all labels, which can lead to a lower ROC AUC.

In contrast, LR and XGBoost produce probability estimates that are more spread out and better calibrated across the full label space. This improves global discrimination between classes, which translates into higher ROC AUC, but comes at the cost of fewer top-ranked predictions, which can slightly reduce LRAP.

5.3 Experiments and future work

Due to computational constraints and limited time, several configurations, such as tuning the chunk window size and testing different preprocessing and feature extraction combinations, could not be fully explored. We also experimented with alternative methodologies that ultimately proved unfeasible within our constraints. We report these attempted approaches and potential directions for future exploration.

5.3.1 Chunk Concatenation

Instead of predicting each chunk independently, another approach is to concatenate sequential chunk features for the model to capture chronological context across the timeline. However, this method introduces two major bottlenecks:

- **Dimensionality:** Depending on the feature set and window size, the resulting vectors become extremely high-dimensional.
- **Temporal Inconsistency:** To enforce a uniform feature length, each audio file must be divided into the same number of chunks. Given the high variance in audio durations, the window size would vary from file to file, meaning the information density per chunk would not be consistent.

To mitigate this, one could split the audio into fixed windows and keep only the k most pertinent chunks. However, selecting these k chunks is non-trivial; we cannot simply filter out silence, as silence itself often carries critical contextual information. Furthermore, this approach is inherently incompatible with the soundscape dataset, where the audio is already strictly pre-segmented into 5-second intervals.

5.3.2 Chunk Features Summary

To capture temporal variations without exploding the feature space, we proposed feature summaries across chunks. This approach follows a two-step aggregation process:

1. **Local feature extraction:** For each individual chunk, we compute our features.
2. **Global feature computation:** We then recompute a second layer of features across all the chunks' local features.

This design allows us to model how specific acoustic features vary across the entire duration of the recording. By aggregating information this way, the model can capture macro-temporal patterns like the rhythm and cadence of the species. But this method also encounters the problem of soundscapes

5.3.3 Hierarchical Two-Stage Classifier

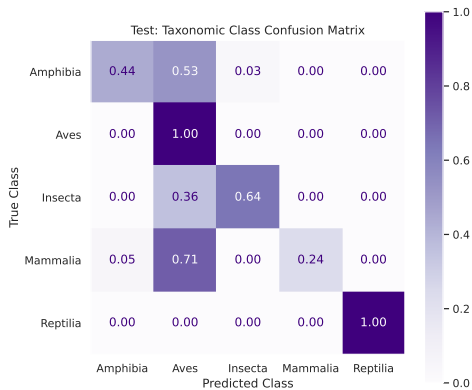


Figure 28: Taxonomic class confusion matrix.

The next method we envisioned is a two-stage classification architecture. The first stage would predict the broad taxonomic class. If a class is detected, the second stage would trigger OVR models specific to the species within that class, allowing each sub-model to specialize in fine-grained discrimination.

An SVC was briefly evaluated for the first stage as an initial exploration. Figure 28 shows the resulting class-level confusion matrix. Mammalia and Amphibia are mainly predicted as Aves, meaning the first stage fails to provide a reliable step to the second stage. To have better results, future work includes resolving the class imbalance at the first stage through class-weighted loss, oversampling of minority classes, or a dedicated binary detector per taxonomic class.

6 Conclusion

In conclusion, this project presented a challenging machine learning task from audio recordings characterised by severe class imbalance, noisy soundscapes, and over 200 species. Working with classical machine learning methods, we designed an audio feature pipeline combining MFCCs, spectral, and temporal descriptors, evaluated across two main strategies.

The global feature-based strategy, aggregating full recordings into a single vector, achieved a top-1 accuracy of 0.4999 and AP of 0.4425 with SVC on the audio test set, but generalised poorly to soundscapes. The chunk aggregation strategy, dividing recordings into overlapping 5-second windows and aggregating predictions via max-pooling, better captured temporal structure. XGBoost emerged as the strongest model (top-1: 0.4572, AP: 0.3737) though at the cost of even stronger Aves bias. In both strategies, minority taxonomic classes (Mammalia, Amphibia, Reptilia) collapsed regularly into Aves, as confirmed by confusion matrices, PR curves, and per-species AP distributions. Furthermore, the AUC–AP discrepancy (ROC AUC $\approx$ 0.92 vs. macro AP $\approx$ 0.42) highlighted that high ranking ability does not translate to precise species-level predictions under imbalance.

Future work and improvement include a better baseline comparability (such as a random classifier to have a better understanding of our LRAP), class-imbalance mitigation through weighted losses, oversampling, stronger feature representations, and the hierarchical two-stage architecture, whose first stage currently collapses due to the same imbalance issues. Neural approaches such as CNNs operating directly on spectrograms would likely yield the most significant performance gains and represent the natural next step beyond the constraints of this project.

7 Annexe

7.1 Grid search best parameters

Here we add the best hyperparameters found for each model after performing random grid search.

SVC		Logistic Regression	
Parameter	Value	Parameter	Value
kernel	rbf	C	1.0
C	10.0	class_weight	balanced
class_weight	balanced	solver	lbfgs
probability	True	max_iter	2000
gamma	scale	random_state	42
max_iter	500 ¹		
random_state	42		

Table 7: Best hyperparameters for SVC and LR.

XGBoost	
Parameter	Value
learning_rate	0.1
max_depth	7
min_child_weight	5
subsample	0.8
colsample_bytree	0.8
reg_alpha	0.1
reg_lambda	1.0
max_delta_step	1
max_bin	64
tree_method	hist
device	cuda
n_estimators	100
eval_metric	logloss
verbosity	0
random_state	42

Table 8: Best hyperparameters for XGBoost.

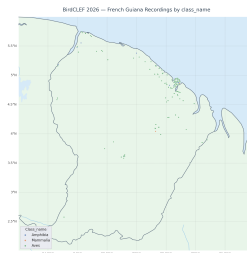


Figure 29: French Guyana recordings location and class

¹ For the chunking aggregation strategy, we used `max_iter = 50`